DevOps

Lab 4: Simple Python Database Connections



Student	MALAYKA IBRAR (058)
Server prompt	malayka-ibrar@Ubuntu
Databases fork	https://github.com/malayka-ibrar/Databases
Submission repository	https://github.com/malayka-ibrar/DevOps

Task 1: Fork and clone the Databases repository

Forked the Databases repository on GitHub. Then cloned my own fork to the Ubuntu server.



Figure: **databases_fork.png**. My fork on GitHub (malayka-ibrar/Databases) with the terminal next to it.

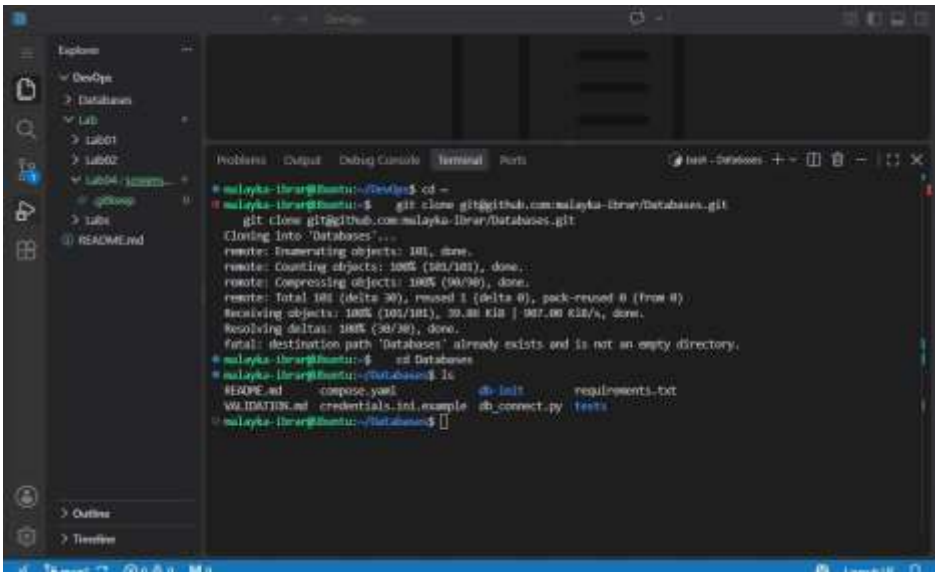


Figure: **databases_clone.png**. The clone of my fork on the server. The files are listed.

Task 2: Install Docker, Docker Compose and Python

Installed Docker, the Docker Compose plug-in, Python and a virtual environment. Then signed out, signed in again and checked the versions.

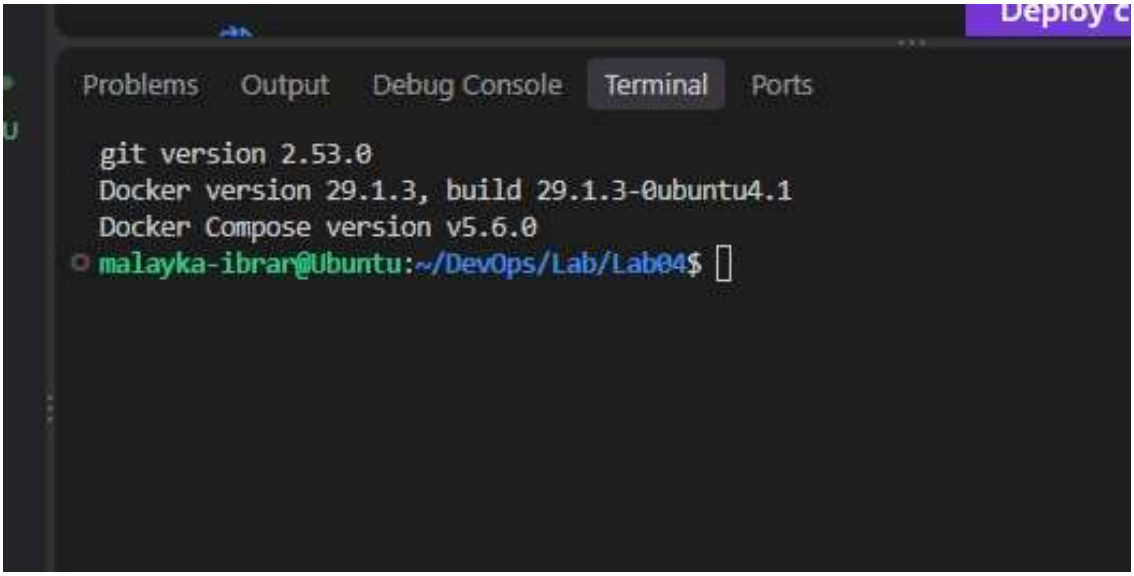


Figure: **docker_installation.png**. Git, Docker and Docker Compose are installed.

Task 3: Start the database services

Started MySQL and three MongoDB databases (dev, test and prod) with Docker Compose. Then I checked the ports.

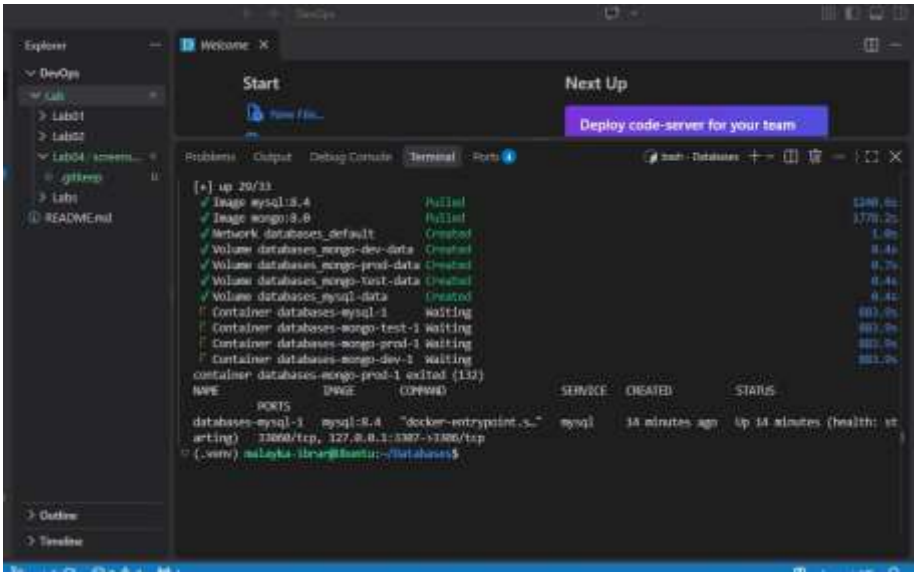


Figure: **database_services.png**. The four database services.

```
docker compose port mysql 3306
docker compose port mongo-dev 27017
docker compose port mongo-test 27017
docker compose port mongo-prod 27017
```

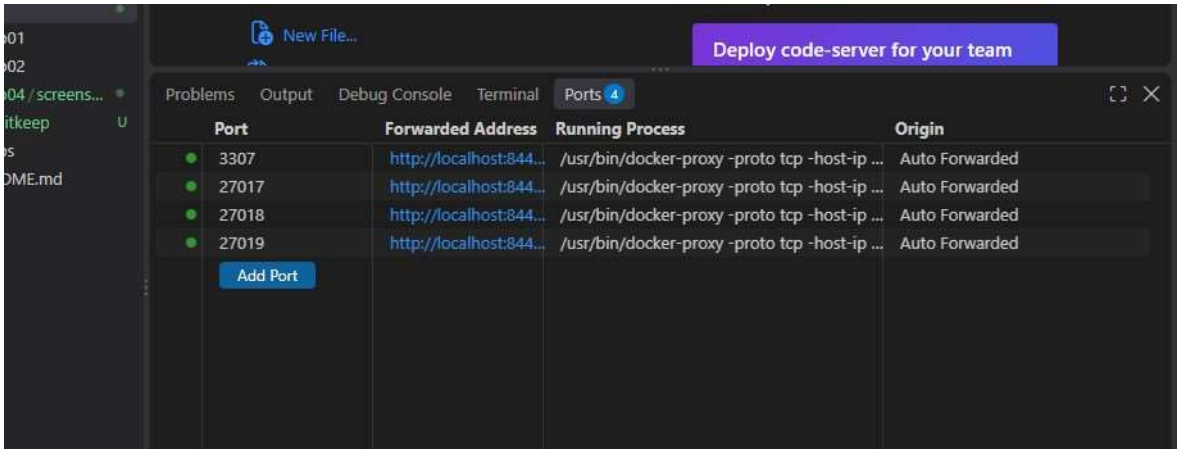


Figure: **database_ports.png**. The ports: MySQL 3307, dev 27017, test 27018, prod 27019.

Task 4: MySQL with values in the code

The username, password, host, port and database name are written inside the Python file. The program connects, asks for the environment name and prints it. `python db_connect.py`

Result: Connected to MySQL: school_mysql. Server 127.0.0.1 3307. Environment: dev.

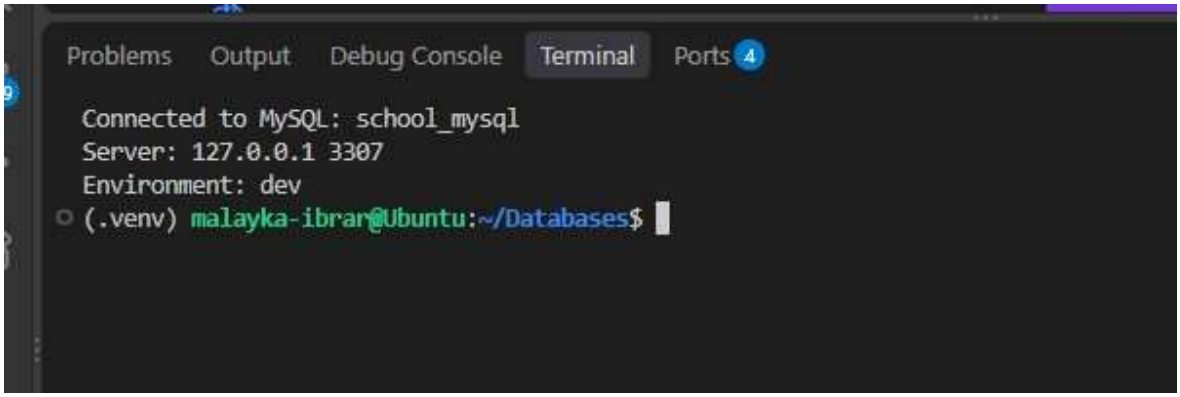


Figure: **mysql_hardcoded_connection.png**. MySQL connection with values written in the code.

Task 5: MongoDB with values in the code

The username, password, endpoint and database name are separate variables in the file. Python joins them to make the connection address (URI). The program sends a ping to prove the connection works. `python db_connect.py`

Result: Connected to MongoDB: school_dev. Server port 27017. Environment: dev.

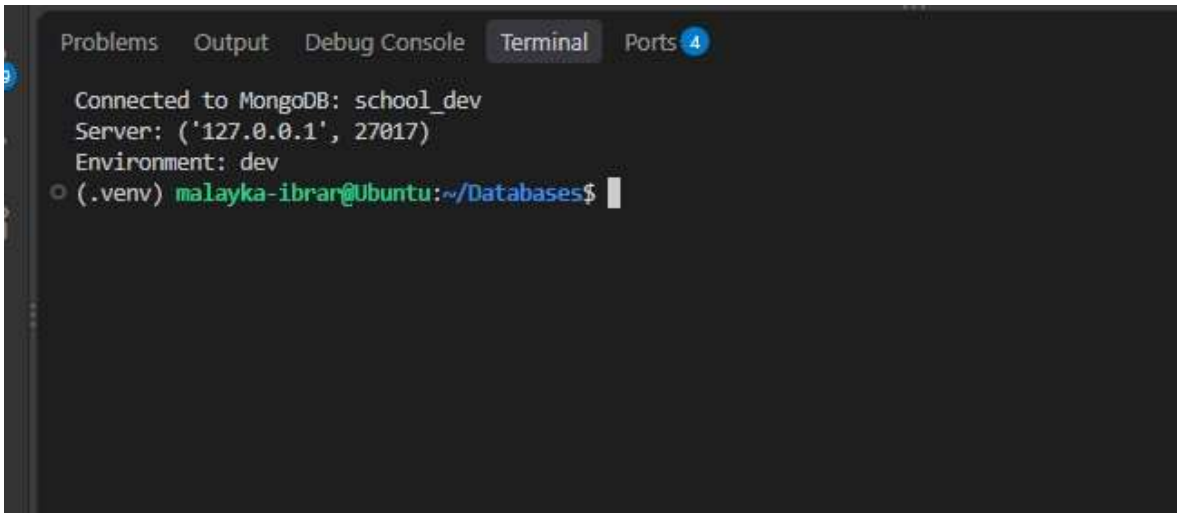


Figure: **mongo_hardcoded_connection.png**. MongoDB connection with values written in the code.

Task 6: MongoDB with os.getenv()

Now the program reads four values from environment variables: MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT and MONGO_DATABASE. The code stays the same. Only the variables change. Exported the values for each environment (values hidden here), cleared the screen, and ran the program.

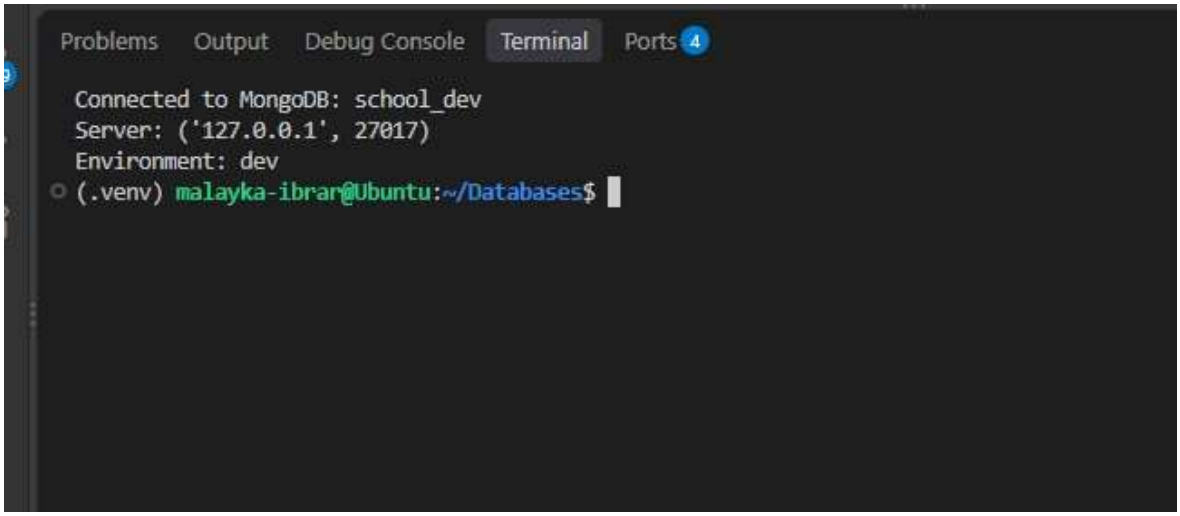


Figure: **mongo_env_dev.png**. Dev: school_dev, port 27017, environment dev.



Figure: **mongo_env_test.png**. Test: school_test, port 27018, environment test.

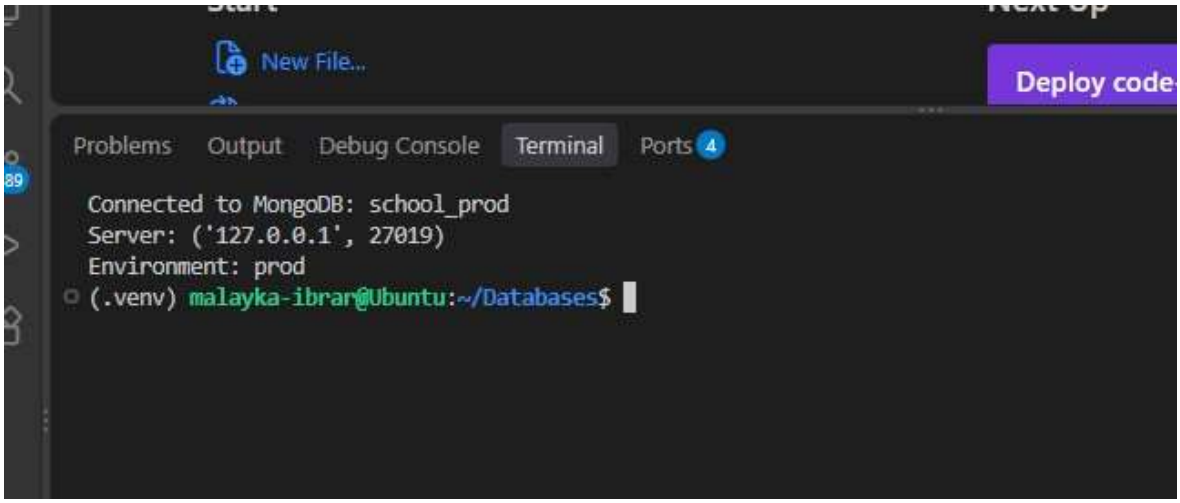


Figure: **mongo_env_prod.png**. Prod: school_prod, port 27019, environment prod.

Task 7: MongoDB with a credentials file

Made a local file named credentials.ini from the example file. It has three sections: dev, test and prod. Limited the file to my account only (mode 600). The file is not committed to Git.



Figure: **credential_files_created.png**. The credentials.ini file was created.

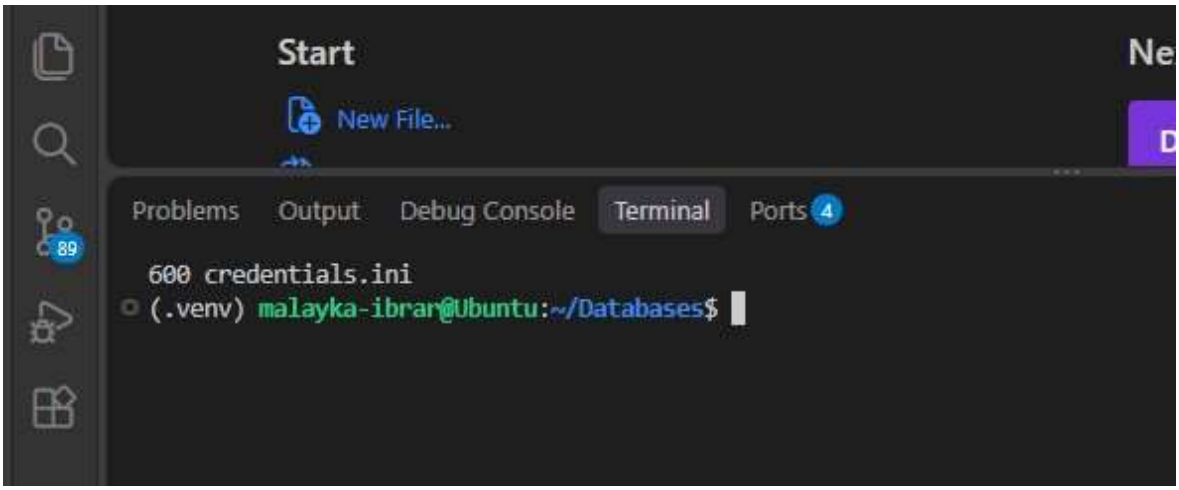


Figure: **credential_file_permissions.png**. The file mode is 600, so only I can read and write it.

The new code reads the section name from the APP_ENV variable. If APP_ENV is not set, it uses dev.

```
export APP_ENV=dev   (then test, then prod)
python db_connect.py
```

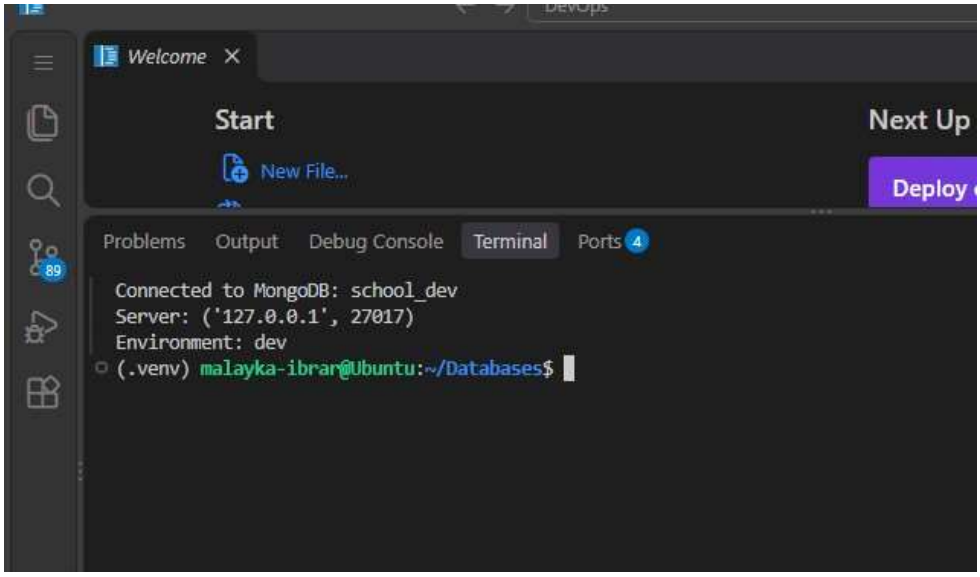


Figure: **mongo_file_dev.png**. APP_ENV=dev: school_dev, port 27017.

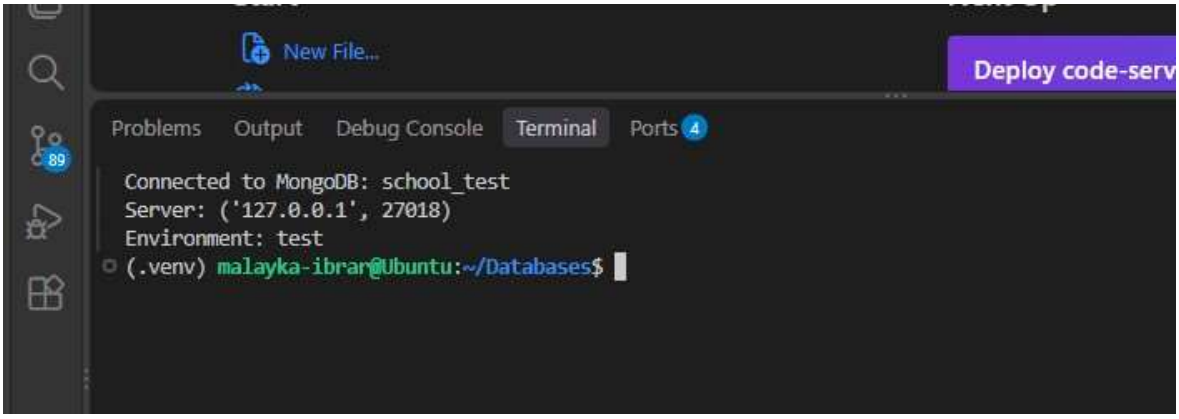


Figure: **mongo_file_test.png**. APP_ENV=test: school_test, port 27018.

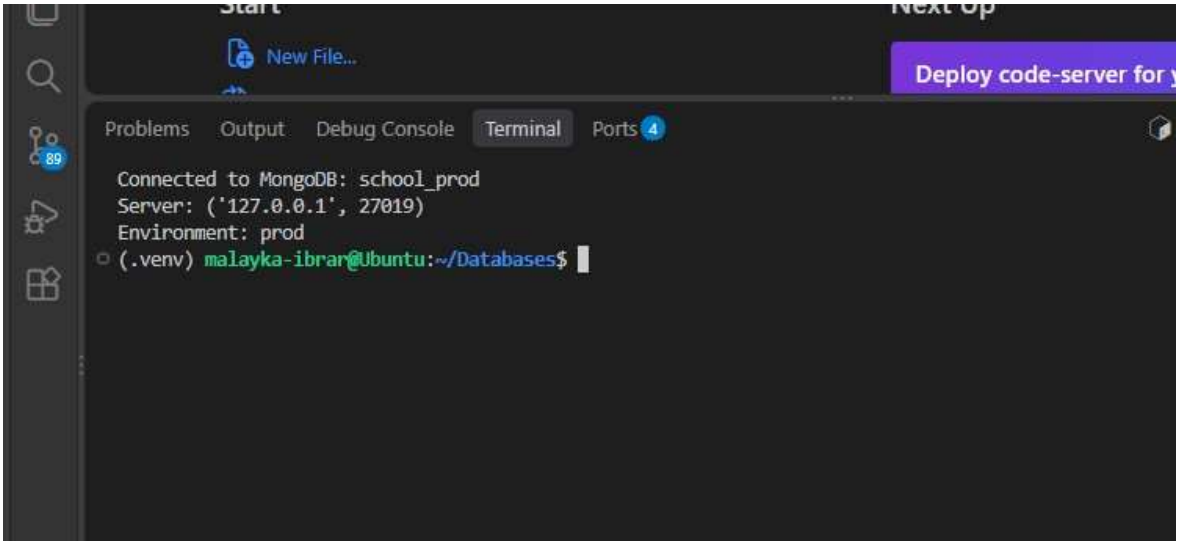


Figure: **mongo_file_prod.png**. APP_ENV=prod: school_prod, port 27019.

Exam questions

Question 1: A missing environment variable

Prediction: the program will stop and print a message.
What I did: I kept the prod values, then removed only MONGO_DATABASE with `unset MONGO_DATABASE` and ran the program.

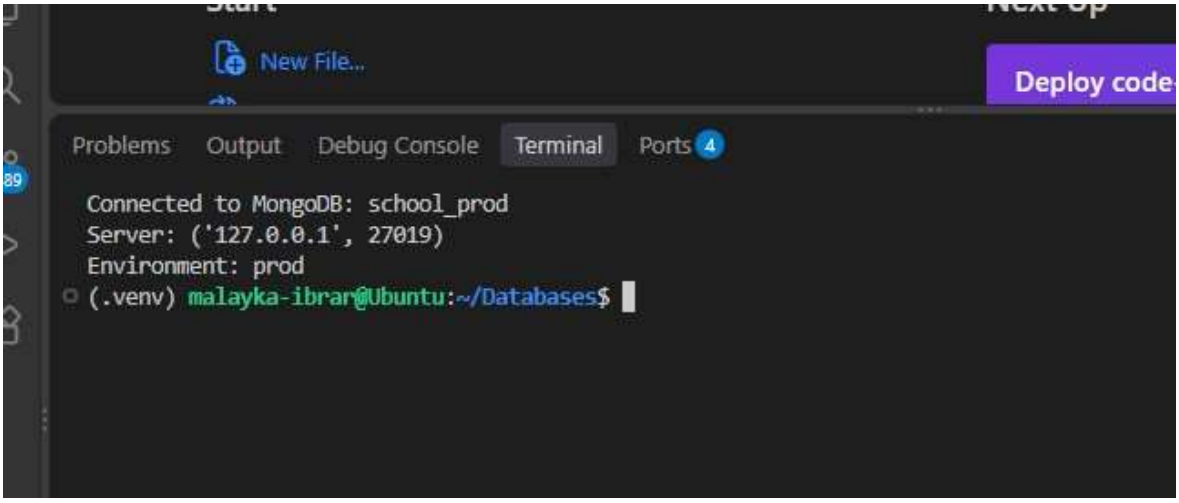


Figure: **Q1_missing_environment.png**. The program stops with: Set all four MongoDB environment variables first.

Explanation: `os.getenv("MONGO_DATABASE")` returned nothing (None) because the variable was removed. The `if` line checks that all four values exist. One was empty, so the program stopped with SystemExit. It stopped before it tried to connect. After I exported MONGO_DATABASE=school_prod again, the connection worked.

Question 2: Which source does the file version read?

What I did: the prod variables from Task 6 were still in my shell. I removed only APP_ENV with `unset APP_ENV` and ran the program.

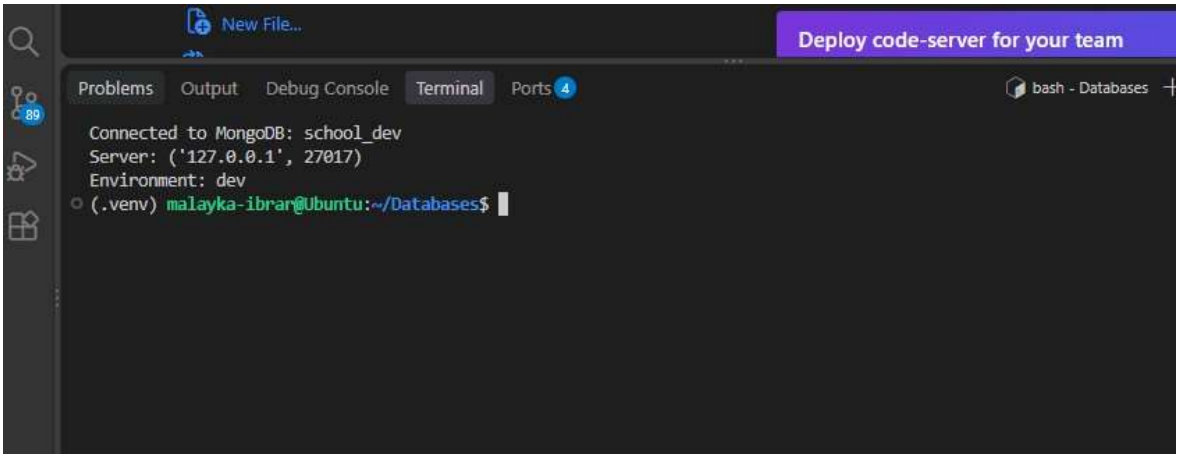


Figure: **Q2_file_source_selection.png**. The program connected to dev.

Explanation: the file version does not read the MONGO_ variables at all. It reads only APP_ENV to choose a section in credentials.ini. When APP_ENV is not set, the code uses the default value dev, so it used the [dev] section. Then I set APP_ENV=test and the test connection worked.

Question 3: A wrong password in the file

What I did: in the [test] section only, I changed the password to *wrong*. Then I ran the program with APP_ENV=test.



Figure: **Q3_wrong_file_password.png**. The program fails with an authentication error.

Explanation: the endpoint only tells Python where the database is. The database server still checks the username and password. A wrong password is rejected, even when the address and

port are right. After the test, Put the correct dummy password back and the connection worked again. Did not commit the credentials file.